The **Transformer architecture** is the foundational model behind modern NLP systems like BERT, GPT, and T5. It was introduced in the groundbreaking 2017 paper:

> **"Attention is All You Need"** by Vaswani et al.

It revolutionized NLP by **removing recurrence (RNNs/LSTMs)** and instead relying **entirely on attention mechanisms** to model relationships in sequential data.

🧠 Overview of Transformer Architecture

The Transformer is composed of:

- An **Encoder** (processes the input)
- A **Decoder** (generates output, e.g., in translation)

Each consists of **repeated blocks (layers)** made of:

- Multi-head self-attention mechanisms
- Feed-forward neural networks
- Residual connections
- Layer normalization

🔧 Core Concepts

1. **Input Representation**

- Input is tokenized and converted into vectors via **embedding layers**.
- **Positional encoding** is added because the model is not sequential (like RNNs) and has no sense of order.

> 🧠 Example: Word embedding + position encoding = input vector

---

2. **Self-Attention Mechanism**

This is the heart of the Transformer. It allows each token to "attend" (look at) all other tokens and weigh their importance.

⚙️ **How It Works:**

For each token, compute:

- **Query (Q)**
- **Key (K)**
- **Value (V)**

Then compute:

$$ \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V $$

- This gives a **weighted sum** of all tokens in the sequence.
- The score QK^T measures the similarity between tokens.

---

3. **Multi-Head Attention**

Instead of using just one attention head, we use multiple in parallel:

```
[Head1] [Head2] ... [HeadN] → Concatenated → Linear Layer
```

- Allows the model to capture different types of relationships (e.g., syntax, long-distance dependencies).
- Each head has its own Q, K, V projection.

---

4. **Position-wise Feed Forward Network (FFN)**

Each token vector is independently passed through a small MLP:

$$ \text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 $$

- Adds non-linearity and expressiveness after attention.

---

5. **Residual Connections & Layer Normalization**

Every sub-layer (attention, FFN) is wrapped in:

```
LayerNorm(x + Sublayer(x))
```

- Prevents vanishing gradients
- Helps train deep networks
- Makes learning more stable

🏗️ Full Encoder Block

```
Input → Embedding → Positional Encoding →
  ┌──────────────────────────────────────────┐
  │ 1. Multi-Head Self-Attention             │
  │ 2. Add & Norm                            │
  │ 3. Feed Forward Network (FFN)            │
  │ 4. Add & Norm                            │
  └──────────────────────────────────────────┘
 → Output to next encoder layer (stacked N times)
```

Full Decoder Block

The decoder is similar but with extra steps:

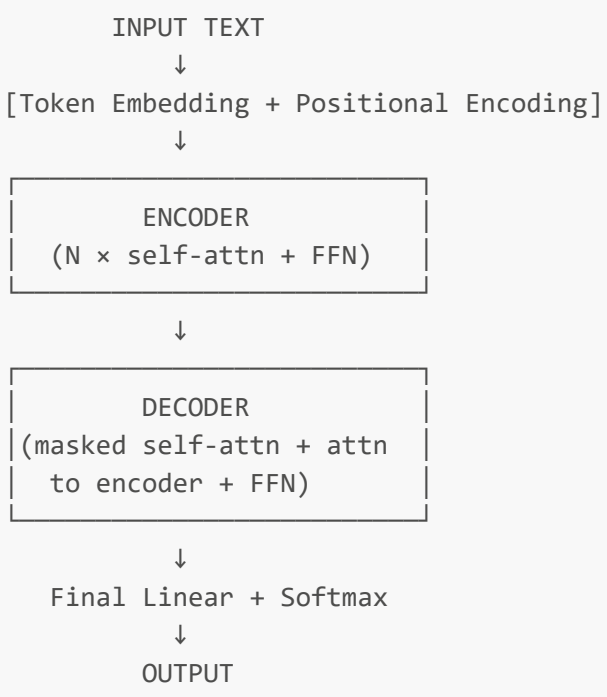
```
Input → Embedding → Positional Encoding →
  ┌──────────────────────────────────────────────────────────┐
  │ 1. Masked Multi-Head Self-Attention (prevents peeking ahead) │
  │ 2. Add & Norm                                            │
  │ 3. Multi-Head Attention over Encoder Output              │
  │ 4. Add & Norm                                            │
  │ 5. Feed Forward Network (FFN)                            │
  │ 6. Add & Norm                                            │
  └──────────────────────────────────────────────────────────┘
 → Output
```

> Masked attention ensures autoregressive behavior (token at position t can't see tokens at t+1).

Transformer Variants

Model	Description
BERT	Uses only the encoder. Great for classification, NER, QA (non-autoregressive).
GPT	Uses only the decoder. Autoregressive, great for text generation.
T5 / BART	Uses encoder-decoder. Suitable for translation, summarization, QA.

Diagram (Text Form)



🔍 Why It Works So Well

- **Parallelism**: Unlike RNNs, Transformers can process all tokens at once.
- **Long-range dependencies**: Self-attention allows every token to look at every other token directly.
- **Scalability**: Easy to scale up with more layers and heads.

🧠 Summary

Component	Function
Self-Attention	Capture relationships between words
Multi-head	Multiple attention patterns
FFN	Non-linear transformation per token
Positional Enc.	Inject sequence order
Residual + Norm	Stabilize and speed up training
Masking	Enforce autoregressive decoding (for generation)